

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных *наук*

Кафедра программной инженерии в информационных системах

Разработка мобильного приложения интернет-магазина электроники на
платформе Android

Курсовая работа по дисциплине
Разработка мобильных приложений
6 семестр 2025/2026 учебного года
09.03.02 Информационные системы и технологии

Зав. кафедрой _____ д. ф.-м. н., доцент С.Д. Махортов

Обучающийся _____ ст. 3 курса В.Е. Ерохов

Руководитель _____ В.А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ, СОКРАЩЕНИЯ	3
ВВЕДЕНИЕ	4
1 Постановка задачи.....	7
1.1 Обзор аналогов	8
1.1.1 Приложение Wildberries	9
1.1.2 Приложение Ozon.....	10
2 Анализ предметной области	13
2.1 Теоретические аспекты предметной области	13
3 Реализация	16
3.1 Средства реализации	16
3.2 Логика приложения	18
3.2.1 Архитектура приложения	22
3.3 Реализация интерфейса	23
3.3.1 Блок-схема алгоритма	28
3.3.2 Диаграмма вариантов использования	30
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	34

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ, СОКРАЩЕНИЯ

В настоящей курсовой работе применяются следующие термины с соответствующими определениями.

BaaS (Backend-as-a-Service) – модель предоставления серверной инфраструктуры как услуги, позволяющая разработчику сосредоточиться на клиентской части.

Clean Architecture – архитектурный паттерн Р. Мартина, разделяющий приложение на независимые концентрические слои с направлением зависимостей строго внутрь.

DI (Dependency Injection) – внедрение зависимостей; паттерн проектирования, при котором зависимости объекта передаются извне.

JWT (JSON Web Token) – компактный формат токена для аутентификации и авторизации, передающий данные в виде JSON-объекта.

ВВЕДЕНИЕ

В современном мире мобильные технологии занимают ключевое место в повседневной жизни. Смартфоны стали основным инструментом для совершения покупок, общения, получения услуг и потребления информации. По данным аналитических агентств, доля мобильной электронной коммерции (m-commerce) в общем объёме онлайн-продаж превышает 50% и продолжает расти. Пользователи всё чаще предпочитают совершать покупки через специализированные мобильные приложения, а не через веб-браузеры, поскольку приложения обеспечивают более высокую скорость работы, удобный интерфейс и дополнительные возможности — такие как push-уведомления и работа в офлайн-режиме.

Платформа Android занимает доминирующее положение на рынке мобильных операционных систем с долей более 70% мирового рынка. Это делает разработку качественных Android-приложений приоритетным направлением для бизнеса в сфере электронной торговли. При этом современные пользователи предъявляют высокие требования к мобильным приложениям: они ожидают мгновенного отклика интерфейса, синхронизации данных в режиме реального времени, интуитивной навигации и надёжной защиты персональных данных.

Настоящая курсовая работа посвящена разработке мобильного приложения интернет-магазина для платформы Android. Приложение реализует полный цикл взаимодействия покупателя с магазином: от регистрации и аутентификации до оформления заказа и просмотра его истории. Для хранения данных и обеспечения серверной логики используется облачная платформа Supabase, предоставляющая базу данных PostgreSQL, модуль аутентификации и механизм realtime-подписок на изменения данных [1] [2].

Актуальность темы обусловлена стремительным ростом рынка мобильной электронной коммерции и высоким спросом на качественные

Android-приложения для онлайн-торговли. Использование архитектурных паттернов Clean Architecture и MVVM позволяет создать приложение с высокой поддерживаемостью кода, лёгким масштабированием и возможностью расширения функциональности без значительного рефакторинга [3].

Целью курсовой работы является проектирование и реализация мобильного Android-приложения интернет-магазина, поддерживающего полный цикл взаимодействия покупателя с магазином. Для достижения цели необходимо решить следующие задачи:

- изучить предметную область мобильной разработки на платформе Android и электронной коммерции;
- проанализировать современные архитектурные подходы и обосновать выбор паттерна;
- спроектировать трёхслойную архитектуру приложения согласно принципам Clean Architecture;
- реализовать функциональные модули: аутентификация, каталог, корзина с realtime, заказы, профиль;
- разработать пользовательский интерфейс в соответствии с Material Design с поддержкой двух языков;
- выполнить интеграцию с Supabase для хранения данных, аутентификации и realtime-функциональности.

Объектом исследования является процесс разработки мобильного приложения для платформы Android. Предметом исследования — инструменты, паттерны и технологии, применяемые при создании Android-приложений в сфере электронной коммерции.

Практическая значимость работы заключается в создании функционально завершённого приложения, демонстрирующего применение современных архитектурных паттернов и технологий в реальном проекте.

Структура курсовой работы включает введение, три основных раздела, заключение и список использованных источников.

1 Постановка задачи

Целью данной курсовой работы является разработка мобильного приложения интернет-магазина для платформы Android, обеспечивающего покупателям удобный доступ к каталогу товаров и возможность оформления заказов непосредственно с мобильного устройства.

Анализ существующих решений в области мобильной электронной коммерции позволяет выделить ключевые требования к разрабатываемому приложению. Конкурентные приложения — Wildberries, Ozon, AliExpress — реализуют схожий функциональный набор: каталог с поиском, корзину, оформление заказов и историю покупок. Ключевыми факторами пользовательского опыта являются скорость работы, актуальность данных и простота навигации.

Разрабатываемое приложение должно обеспечивать следующие функциональные требования.

Требования к модулю аутентификации:

- регистрация нового пользователя с указанием имени, телефона, email и пароля;
- валидация данных на стороне клиента: формат email, формат телефона, сложность пароля;
- проверка уникальности номера телефона в базе данных перед регистрацией;
- аутентификация по email и паролю; сохранение сессии между запусками; безопасный выход.

Требования к модулю каталога:

- отображение всех товаров в виде сетки с карточками (название, бренд, цена, изображение);
- поиск товаров по названию в реальном времени при каждом изменении строки поиска;

- сортировка по цене (возрастание/убывание) и по названию (А–Я / Я–А);
- отображение текущего количества товара в корзине непосредственно на карточке.

Требования к модулю корзины:

- добавление товара; при повторном добавлении — увеличение количества;
- уменьшение количества; при достижении нуля — удаление из корзины;
- синхронизация состояния корзины в реальном времени;
- отображение итоговой стоимости; выбор способа оплаты; оформление заказа.

Требования к модулю заказов: история всех заказов пользователя с детализацией по составу, статусу и способу оплаты.

К нефункциональным требованиям относятся:

- безопасность: пароли хранятся только в хешированном виде; доступ через JWT;
- поддержка русского и английского языков через ресурсные файлы Android;
- соответствие Material Design; устойчивость к повороту экрана;
- минимальная версия Android: API 24 (Android 7.0 Nougat).

1.1 Обзор аналогов

Перед формулировкой требований к разрабатываемому приложению был проведён анализ существующих мобильных приложений в сфере электронной коммерции. Для анализа выбраны два наиболее распространённых приложения на российском рынке: Wildberries и Ozon. Оба доступны на платформе Android и представляют значительную долю рынка. Анализ позволяет выявить функциональные стандарты отрасли, определить

сильные стороны для учёта при проектировании и зафиксировать недостатки, которых следует избежать.

1.1.1 Приложение Wildberries

Wildberries — крупнейший маркетплейс в России и СНГ. Мобильное приложение входит в топ загружаемых в Google Play в категории «Покупки». Поддерживает Android 6.0 и выше.

Функциональные возможности:

- каталог с глубокой категоризацией (50+ категорий), фильтрацией по цене, бренду, рейтингу, цвету, размеру;
- полнотекстовый поиск с автодополнением, исправлением опечаток и поиском по фотографии;
- корзина с выбором пункта самовывоза или доставки на дом, расчётом стоимости доставки;
- история заказов, отслеживание доставки на карте в реальном времени;
- система лояльности: кэшбэк, купоны, персональные скидки;
- отзывы и оценки с фото и видео; поддержка нескольких способов оплаты.

Достоинства:

- высокая скорость при больших каталогах за счёт агрессивного кеширования;
- богатая система фильтров, позволяющая точно сузить поиск;
- развитая инфраструктура доставки с пунктами на карте;
- персонализированные рекомендации на основе истории просмотров.

Недостатки:

- перегруженный интерфейс: баннеры и акции затрудняют навигацию;
- большой объём приложения (более 150 МБ);

- агрессивные push-уведомления по умолчанию;
- UI-паттерны не всегда соответствуют Material Design.

В таблице 1 приведена информация приложения Wildberries.

Таблица 1 - Характеристики приложения Wildberries

Параметр	Значение
Платформа	Android 6.0+, iOS 14+
Аутентификация	Телефон + SMS-код, email, соцсети
Поиск	Полнотекстовый, по фото, автодополнение
Синхронизация корзины	Да, в реальном времени
Способы оплаты	Карта, СБП, наличные, кредит, рассрочка
Размер	~150 МБ

1.1.2 Приложение Ozon

Ozon — второй по величине маркетплейс в России. Выделяется продуманным интерфейсом и широкими возможностями персонализации. Поддерживает Android 7.0 и выше.

Функциональные возможности:

- каталог с многоуровневыми фильтрами; поддержка AR-примерки для ряда категорий;
- умный поиск с голосовым управлением, поиском по изображению и предиктивными подсказками;
- корзина с расчётом итоговой суммы с учётом скидок и купонов до оформления;
- Ozon Карта — банковская карта с кэшбэком, интегрированная в приложение;
- отслеживание заказов с историей статусов; возврат через QR-код; чат с продавцом.

Достоинства:

- чистый и современный дизайн, близкий к рекомендациям Material Design;
- прозрачное ценообразование: финальная стоимость видна до оформления заказа;
- развитая система возвратов без звонков в поддержку;
- офлайн-режим: часть контента кешируется для просмотра без интернета.

Недостатки:

- высокое потребление ОЗУ: на устройствах с 2 ГБ возможны замедления;
- сложная структура разделов (маркетплейс + банк + доставка) затрудняет навигацию;
- периодические проблемы с отображением цен при нестабильном соединении.

В таблице 2 приведена информация приложения Ozon.

Таблица 2 - Характеристики приложения Ozon

Параметр	Значение
Платформа	Android 7.0+, iOS 14+
Аутентификация	Телефон + SMS, email, Apple ID, Google
Поиск	Голосовой, по фото, предиктивные подсказки
Синхронизация корзины	Да, с мгновенным пересчётом стоимости
Офлайн-режим	Частичный (кеш каталога)
Размер	~120 МБ

По результатам анализа аналогов обязательными функциями разрабатываемого приложения определены: аутентификация, каталог с поиском и фильтрацией, корзина с синхронизацией, несколько способов

оплаты, история заказов. Приложение намеренно ограничено по масштабу по сравнению с маркетплейсами, что позволяет сосредоточиться на архитектурном качестве кода и демонстрации современных паттернов разработки.

2 Анализ предметной области

Мобильная электронная коммерция (m-commerce) представляет собой совокупность коммерческих операций, осуществляемых посредством мобильных устройств. По оценкам исследовательских компаний, к 2025 году более половины всех онлайн-покупок совершаются с мобильных устройств. Предметная область данной курсовой работы охватывает разработку мобильных приложений для Android и создание ПО для электронной коммерции.

2.1 Теоретические аспекты предметной области

Разработка мобильных приложений для Android имеет ряд принципиальных особенностей, определяющих все ключевые архитектурные решения.

Жизненный цикл Android-компонентов. Activity и Fragment имеют сложный жизненный цикл, управляемый ОС. Система вправе уничтожить компонент при поступлении звонка, нехватке памяти или повороте экрана. Поворот экрана пересоздаёт Activity, что без специальных мер приводит к потере данных и повторным сетевым запросам. Для решения компания Google разработала ViewModel — компонент, переживающий изменения конфигурации и предоставляющий данные через LiveData. Данная связка обеспечивает реактивное обновление UI: при изменении данных в ViewModel интерфейс обновляется автоматически [4] [5].

Асинхронное программирование обязательно для Android: главный поток не должен блокироваться длинными операциями под угрозой ANR (Application Not Responding). Современным стандартом являются Kotlin Coroutines — они позволяют писать асинхронный код в последовательном стиле. Структурированный параллелизм через viewModelScope автоматически отменяет все запущенные корутины при уничтожении ViewModel [6].

Паттерн Repository является стандартным способом абстрагирования источников данных. Repository предоставляет единый интерфейс для получения данных вне зависимости от их источника — сети, локальной БД или кеша. ViewModel работает с репозиторием через интерфейс, что позволяет легко подменять реализацию при тестировании.

Электронная коммерция: ключевые сущности и процессы. Основные сущности: пользователь, товар, корзина, заказ. Процесс покупки: выбор товаров → корзина → оформление заказа → статус заказа. Критически важна атомарность оформления: создание заказа, перенос товаров и очистка корзины должны выполняться как единая операция. При ошибке на любом шаге все изменения должны быть отменены. Фиксация цены на момент заказа — ключевое бизнес-требование: покупатель должен заплатить именно ту сумму, которую видел в корзине.

Синхронизация данных в режиме реального времени важна для корзины: изменения, сделанные с другого устройства, должны немедленно отражаться в текущем сеансе. Для этого применяются технологии WebSocket и SSE. Supabase реализует realtime через PostgreSQL LISTEN/NOTIFY: изменения строк таблицы генерируют события, передаваемые подписанным клиентам по WebSocket.

Безопасность данных. Пароли хранятся исключительно в хешированном виде (bcrypt/argon2). Доступ к API обеспечивается JWT-токенами с ограниченным сроком действия и механизмом refresh. Весь трафик шифруется через HTTPS. Supabase Auth реализует все перечисленные требования из коробки.

Архитектурный паттерн MVVM предполагает три роли: Model (данные и бизнес-логика), View (Activity/Fragment, только отображение), ViewModel (логика представления, данные через LiveData). ViewModel не имеет ссылки на View, что делает его независимым от жизненного цикла и удобным для тестирования.

Clean Architecture расширяет MVVM слоем Domain с Use Case. Use Case инкапсулируют отдельные бизнес-сценарии и не зависят от Android-фреймворка. Это позволяет заменить Supabase на другой бэкенд, не затрагивая логику ViewModel и Use Case — достаточно создать новую реализацию Repository.

Принцип внедрения зависимостей (DI) создаёт слабосвязанные компоненты. Вместо создания зависимостей внутри класса они передаются извне. В Android-экосистеме популярны Hilt (официально рекомендован, основан на Dagger) и Koin (лёгкий альтернативный вариант на Kotlin без кодогенерации) [7] [8].

Локализация обеспечивается встроенным механизмом Android: строки хранятся в XML-файлах в values/ (английский) и values-ru/ (русский). При запуске приложение автоматически выбирает ресурсы на языке системных настроек устройства.

Паттерн Builder применяется для создания сложных объектов через последовательность шагов с читаемым DSL-синтаксисом. В проекте он использован для конфигурации нижней панели навигации: вместо прямого создания объектов разработчик описывает структуру декларативно, что улучшает читаемость и расширяемость кода [9].

Паттерн Repository с интерфейсами позволяет ViewModel работать с абстракцией источника данных, не зная о конкретной реализации. Это обеспечивает подменяемость: для тестов можно создать FakeRepository, возвращающий заданные данные, без обращения к реальному серверу.

Sealed class в Kotlin — это иерархия классов с ограниченным числом наследников, известных компилятору. Result<T> с вариантами Success, Error и Loading обеспечивает исчерпывающую обработку всех состояний через when-выражение без риска пропустить случай.

3 Реализация

В данном разделе описаны средства реализации приложения, его архитектура, логика ключевых компонентов и пользовательский интерфейс. Разработка выполнена на языке Kotlin с использованием Android SDK, набора библиотек Jetpack и облачной платформы Supabase.

3.1 Средства реализации

Для реализации мобильного приложения был выбран современный технологический стек, обеспечивающий высокое качество кода и производительность приложения.

Kotlin — официальный язык Android-разработки. По сравнению с Java предоставляет: null-безопасность на уровне типов, лаконичный синтаксис (data classes, extension functions, лямбды), встроенные корутины, полную совместимость с Java-кодом.

Android SDK предоставляет компоненты: AppCompatActivity и Fragment (основы экранов), RecyclerView с виртуализацией для эффективных списков, ViewBinding для типобезопасного доступа к view-элементам, систему ресурсов (строки, размеры, цвета).

Jetpack Navigation Component обеспечивает декларативную навигацию через граф nav_graph.xml. NavController управляет стеком фрагментов и кнопкой «назад». Подход централизует логику навигации и исключает ручное управление FragmentManager.

Supabase — BaaS на основе PostgreSQL. В проекте используются три модуля: Auth (регистрация, вход, JWT-сессии), PostgREST (REST API над PostgreSQL с фильтрацией и join), Realtime (WebSocket-подписки на изменения таблиц через LISTEN/NOTIFY).

Kotlin Coroutines обеспечивают асинхронность без callback-функций. viewModelScope автоматически отменяет корутины при уничтожении

ViewModel. Dispatchers.IO используется для сетевых запросов, Dispatchers.Main — для обновления UI.

Koin — DI-фреймворк без кодогенерации. Зависимости объявляются в AppModule: single для синглтонов (репозитории, Supabase-клиент), factory для Use Case, viewModel для ViewModel. Инициализация — в MyApplication.onCreate().

OkHttp — HTTP-клиент для Supabase. Настройки: таймауты 30 с, ping-интервал 20 с для поддержания WebSocket, логирование запросов в debug-режиме.

kotlinx.serialization обеспечивает сериализацию data class в JSON. @Serializable и @SerialName мапят snake_case полей БД в camelCase Kotlin. ignoreUnknownKeys = true защищает от ошибок при добавлении новых серверных полей.

Material Design Components предоставляет TextInputLayout, Snackbar, FloatingActionButton, BottomNavigationView — компоненты с единым визуальным языком Google.

В таблице 3 представлен полный технологический стек.

Таблица 3 - Технологический стек приложения

Компонент	Технология	Назначение
Язык	Kotlin 2.0+	Основной язык разработки
Платформа	Android SDK API 24+	Компоненты приложения
Архитектура UI	ViewModel и LiveData	Управление состоянием экранов
Навигация	Jetpack Navigation	Граф навигации между фрагментами
DI	Koin 3.x	Внедрение зависимостей
База данных	Supabase (PostgreSQL)	Хранение данных, REST API

Аутентификация	Supabase Auth и JWT	Управление сессиями
Realtime	Supabase Realtime	Синхронизация корзины
Асинхронность	Kotlin Coroutines	Неблокирующие операции
HTTP-клиент	OkHttp	Сетевой движок с таймаутами
Сериализация	kotlinx.serialization	Взаимодействие между JSON и Kotlin data class
UI-компоненты	Material Design 3	Стандартные компоненты Google

3.2 Логика приложения

Бизнес-логика реализована согласно принципу единственной ответственности. Каждый класс и каждый метод выполняют строго одну функцию. Результаты всех операций обернуты в sealed class Result<T> с состояниями Success(data), Error(message) и Loading.

Сценарий регистрации: клиент валидирует данные, RPC-функция check_phone_exists проверяет уникальность телефона, Supabase Auth создаёт пользователя, в таблицу users вставляется профиль с привязкой к UUID из Auth. Разделение аутентификационных данных (Auth) и профиля (users) позволяет хранить минимум данных в системе аутентификации.

Сценарий корзины: при добавлении товара запрашивается текущее количество. Если quantity = 0 — INSERT новой записи; иначе — UPDATE с quantity+1. При удалении: если quantity > 1 — UPDATE с quantity-1; иначе или при флаге removeAll — DELETE. Логика реализована в CartRepositoryImpl и полностью скрыта от вышележащих слоёв.

Сценарий оформления заказа (три шага): вычисление суммы и INSERT в orders, получение order_id; для каждого товара INSERT в product_in_order с фиксацией цены; DELETE всех записей пользователя из product_in_cart. Каждый шаг — отдельный запрос; при ошибке ViewModel уведомляет View через errorMessage в UiState.

Диаграмма последовательностей (рисунок 1) ниже демонстрирует механизм realtime-синхронизации данных корзины между различными экранами приложения. При любом изменении в таблице `product_in_cart` (добавление, обновление или удаление товара) система автоматически уведомляет все активные ViewModel, обеспечивая консистентность данных без необходимости ручного обновления.

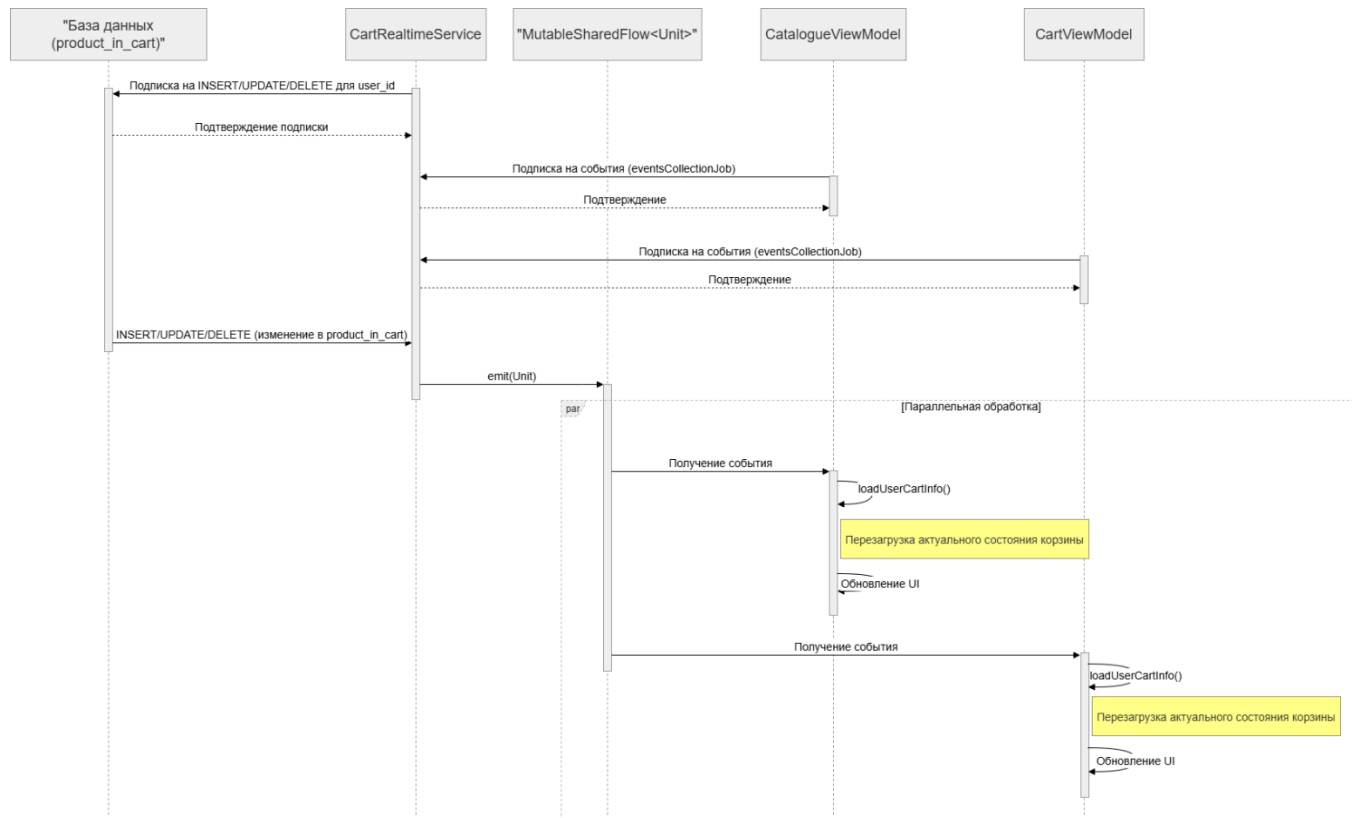


Рисунок 1 – Диаграмма последовательности для механизма realtime-синхронизации

Ниже приведены ключевые фрагменты кода, иллюстрирующие основные архитектурные решения приложения.

Листинг 1 демонстрирует определение sealed class `Result<T>`, используемого для унификации обработки результатов во всех слоях приложения.

Листинг 1 - Sealed class Result для обработки результатов операций

```
sealed class Result<out T> {  
    data class Success<T>(val data: T) : Result<T>()  
    data class Error(val message: String) :  
Result<Nothing>()  
    object Loading : Result<Nothing>()  
}
```

Sealed class Result содержит три варианта: Success с полезными данными типа T, Error с текстом ошибки и Loading для отображения состояния загрузки. Такой подход позволяет обработать все исходы операции в одном when-выражении и исключает необходимость работы с исключениями для управления потоком выполнения.

Листинг 2 показывает пример реализации Use Case — AddToCartUseCase, инкапсулирующего единственный бизнес-сценарий добавления товара в корзину.

Листинг 2 - Use Case добавления товара в корзину

```
class AddToCartUseCase(private val cartRepository:  
CartRepository) {  
    suspend operator fun invoke(  
        userId: Int,  
        productId: Int  
    ): Result<Unit> {  
        return cartRepository.addToCart(userId,  
productId)  
    }  
}
```

Use Case реализован как класс с единственным методом `invoke()`, объявленным через `operator fun` — это позволяет вызывать объект класса как функцию: `addToCartUseCase(userId, productId)`. Тонкая обёртка над репозиторием обеспечивает независимость `ViewModel` от деталей реализации слоя данных.

Листинг 3 иллюстрирует паттерн `UiState`, применяемый в `CartViewModel` для управления состоянием экрана корзины.

Листинг 3 - `UiState` экрана корзины в `CartViewModel`

```
data class CartUiState(  
    val cartUiItems: List<CatalogueUiItem> =  
emptyList(),  
    val isLoading: Boolean = true,  
    val errorMessage: String? = null,  
    val isCreationSuccessful: Boolean = false,  
    val paymentMethod: PaymentMethod =  
PaymentMethod.NOT_CHOSEN,  
    val isInitialized: Boolean = false  
)
```

`UiState` — это `data class`, содержащий всё необходимое для полного рендеринга экрана: список товаров, флаг загрузки, сообщение об ошибке, флаг успешного создания заказа, выбранный способ оплаты и флаг завершения инициализации. `Fragment` наблюдает за единственным `LiveData<CartUiState>` и реагирует на любое изменение состояния.

Листинг 4 показывает фрагмент инициализации `Supabase`-клиента в `AppModule` с настройкой кастомного `OkHttp`-движка.

Листинг 4 - Инициализация `Supabase`-клиента с `OkHttp` в `AppModule`

```
fun createSupabaseClient(): SupabaseClient {
```

```

    val client = OkHttpClient.Builder()
        .connectTimeout(30, TimeUnit.SECONDS)
        .readTimeout(30, TimeUnit.SECONDS)
        .pingInterval(20, TimeUnit.SECONDS)
        .build()

    return createSupabaseClient(supabaseUrl,
supabaseKey) {
        install(Postgrest)
        install(Auth)
        install(Realtime)
        httpEngine = OkHttpClient(okHttpConfig).apply
{
            preconfigured = client })
    }
}

```

Кастомный OkHttpClient с явно заданными таймаутами (30 с) и ping-интервалом (20 с) передаётся в Supabase через OkHttpClient. Ping-интервал особенно важен для поддержания активного WebSocket-соединения realtime-сервиса при отсутствии данных. Три плагина — Postgrest, Auth и Realtime — подключаются декларативно через install().

3.2.1 Архитектура приложения

Архитектура построена по принципам Clean Architecture Р. Мартина. Зависимости направлены строго от внешних слоёв к внутренним: от Presentation к Domain, от Domain к Data. Domain-слой не имеет зависимостей от Android-фреймворка и Supabase.

В таблице 4 представлена архитектура приложения по слоям.

Таблица 4 - Архитектура приложения (Clean Architecture + MVVM)

Слой	Компоненты	Классы	Ответственность
Presentation	Activity, Fragment, Adapter	MainActivity, CatalogueFragment, CartFragment, OrdersFragment, SettingsFragment	Отображение UI, обработка пользовательских событий
ViewModel	ViewModel, LiveData и UiState	CatalogueViewModel, CartViewModel, OrdersViewModel, SettingsViewModel	Управление состоянием экрана, реактивный UI
Domain	Use Case (16 классов)	SignInUseCase, AddToCartUseCase, CreateNewOrderUseCase и др.	Бизнес-сценарии; нет зависимостей от Android-фреймворка
Data	Repository, Service	AuthRepositoryImpl, CartRepositoryImpl, CartRealtimeService	Доступ к Supabase (PostgREST, Auth, Realtime WebSocket)
DI (Koin)	Koin Module	AppModule, MyApplication	Инициализация и связывание зависимостей при старте

На рисунке 2 представлена диаграмма пакетов, которая иллюстрирует модульную архитектуру приложения, реализованную в соответствии с принципами Clean Architecture. Выделены три основных слоя (Data, Domain, Presentation) и модуль внедрения зависимостей (Koin DI). Стрелки показывают направление зависимостей между пакетами, а подписи поясняют тип связи (например, "Uses Repositories", "registers ViewModel").

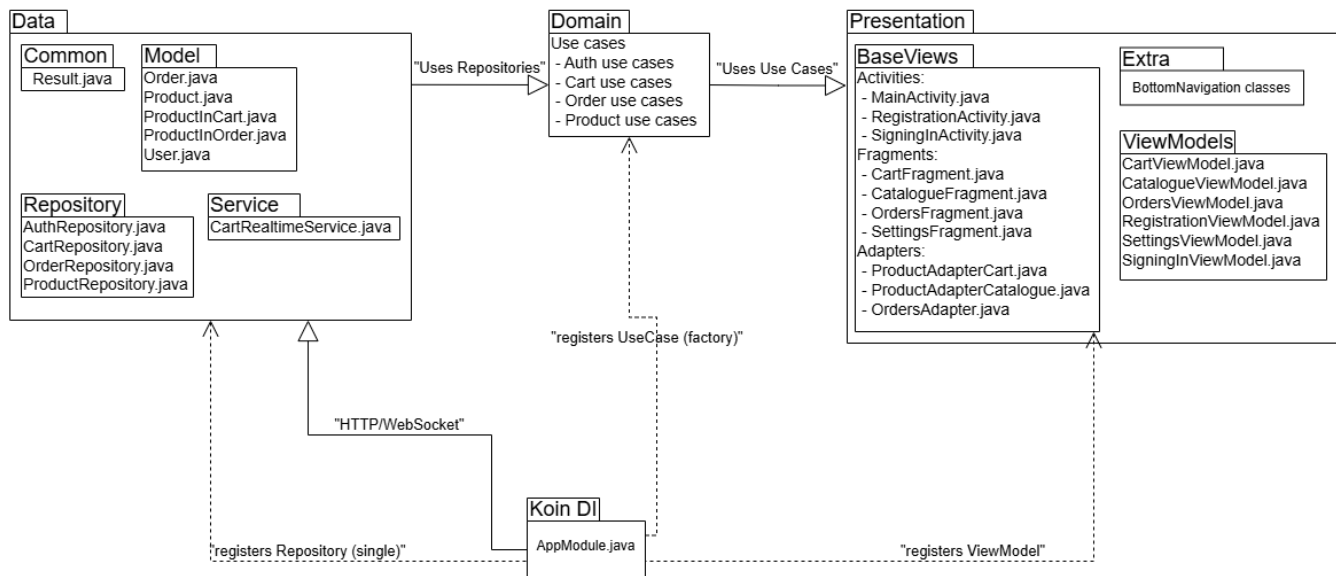


Рисунок 2 – Диаграмма пакетов

3.3 Реализация интерфейса

Пользовательский интерфейс реализован с Material Design 3 в светлой схеме с фиолетовым акцентом. Навигация двухуровневая: Activity-уровень (SignInActivity / RegistrationActivity, далее MainActivity) и Fragment-уровень внутри MainActivity через NavHostFragment.

Нижняя панель навигации настраивается через DSL-Builder: section { title("Catalogue"); iconSource(resource(R.drawable.ic_catalogue)); link("catalogue") }. Это обеспечивает декларативное и читаемое описание навигационной структуры. При нажатии на раздел NavController.navigate(route) переключает текущий Fragment.

На рисунке 3 представлены экраны аутентификации.

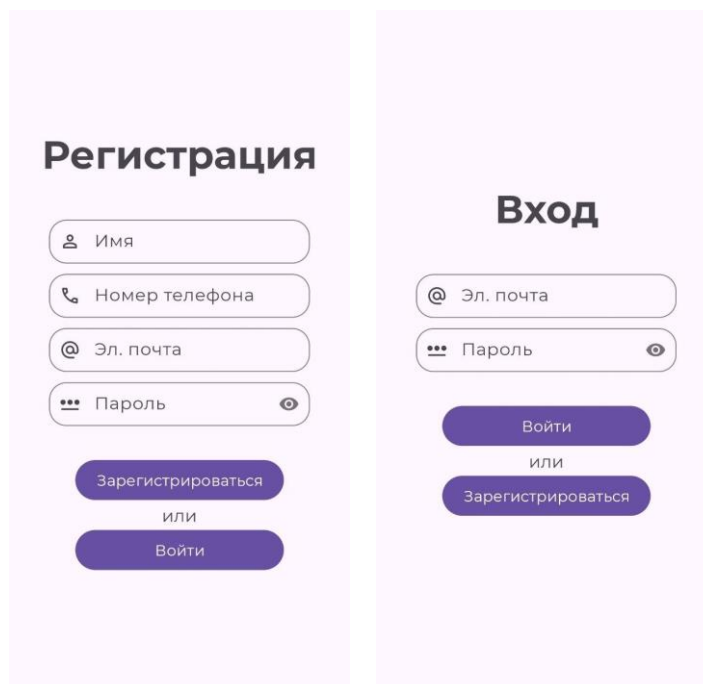


Рисунок 3 - Экран регистрации (слева) и экран входа (справа)

Экран регистрации (`RegistrationActivity`) содержит четыре поля `TextInputLayout`: имя, телефон, email, пароль. Пароль имеет кнопку `toggle` видимости. Валидация при нажатии «Зарегистрироваться»: непустота имени, формат телефона (10 цифр), формат email (regex), сложность пароля (длина, цифра, заглавная, строчная, спецсимвол). При ошибке выводится сообщение под соответствующим полем через `setError()`. Кнопка «Войти» переходит на `SigningInActivity`.

Экран входа (`SigningInActivity`) содержит поля email и пароль. После успешной аутентификации — переход в `MainActivity`. Ошибки: «Неверные данные» при `error=Data`, «Общая ошибка» при `error=General`. При загрузке кнопка «Войти» отображает текст «Загрузка...» и блокируется.

На рисунке 4 представлен экран каталога.

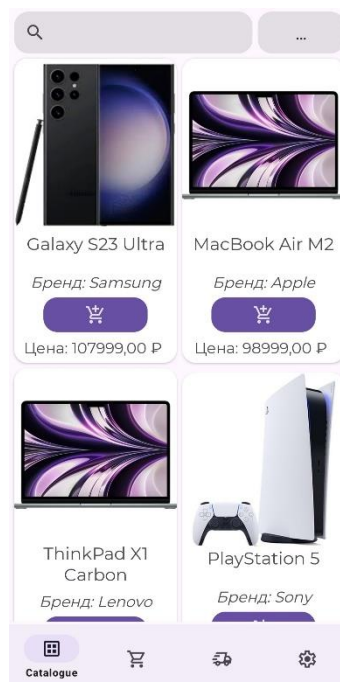


Рисунок 4 - Экран каталога товаров

Экран каталога (CatalogueFragment) отображает товары в GridLayoutManager с 2 колонками. Строка поиска (SearchView) и Spinner фильтров. Поиск работает по onQueryTextChange — фильтрация при каждом символе. Spinner: без фильтра, цена по возрастанию, цена по убыванию, название А–Я, название Я–А. Карточка товара: иконка, название, бренд, цена. При количестве равном нулю — кнопка «добавить в корзину»; при количестве большем нуля — кнопки «+»/«-» со счётчиком. FAB «наверх» появляется при прокрутке (`firstVisiblePosition > 0`) через OnScrollListener.

На рисунке 5 представлен экран корзины.



Рисунок 5 - Экран корзины покупок

Экран корзины (CartFragment) — LinearLayoutManager. Карточка: название, бренд, цена, кнопки $\pm$, счётчик. Итоговая стоимость вычисляется в реальном времени: $\text{sum}(\text{quantity} * \text{price})$ по всем позициям. Spinner оплаты: не выбрано, картой онлайн, картой при получении, наличными, СБП, криптовалюта. Кнопка «Оформить заказ» активна только при `cartUiItems.isNotEmpty() && paymentMethod != NOT_CHOSEN`. При пустой корзине после инициализации — текст «В корзине нет ни одного товара».

На рисунке 6 представлен экран истории заказов.

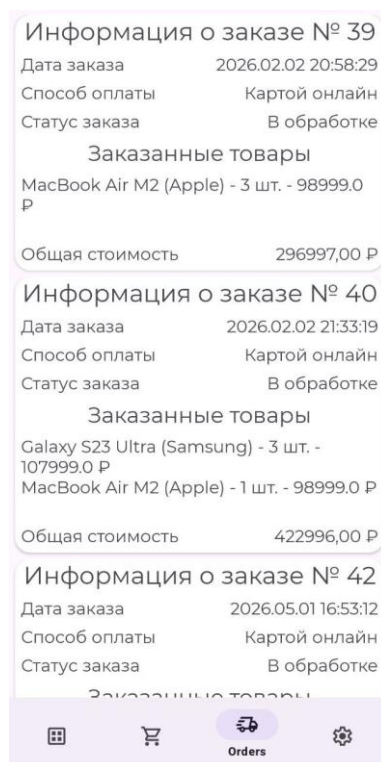


Рисунок 6 - Экран истории заказов

Экран заказов (OrdersFragment) — LinearLayoutManager. Каждая карточка: номер заказа, дата (формат ДД.ММ.ГГГГ ЧЧ:ММ:СС, полученный заменой 'Т' на пробел и '-' на '.'), способ оплаты (из массива строковых ресурсов по коду), статус (10 статусов: «В обработке», «Подтверждён», «Собирается», «Готов к отправке», «Отправлен», «У курьера», «Доставлен», «Отменён», «Возврат», «В пункте выдачи»), список товаров с количеством и ценой, итоговая сумма.

На рисунке 7 представлен экран настроек.

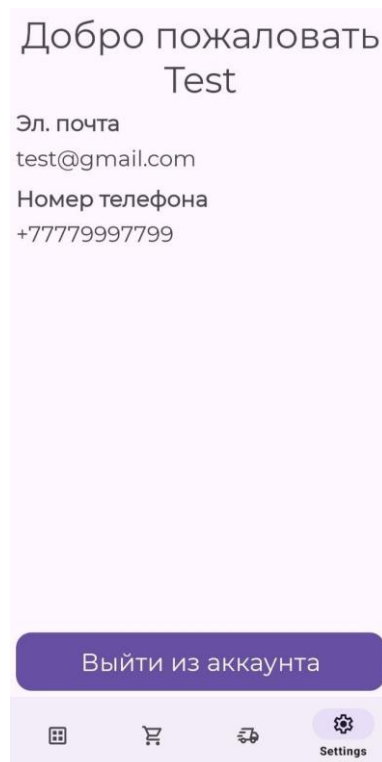


Рисунок 7 – Экран настроек профиля

Экран настроек (SettingsFragment) показывает приветствие «Добро пожаловать [имя]», email и телефон пользователя. Данные загружаются через GetCurrentUserUseCase при инициализации ViewModel. Кнопка «Выйти из аккаунта», SignOutUseCase, при успехе navigateToRegistration().

Все ошибки отображаются через Material Snackbar: для ошибок — красный фон, для успехов — зелёный. Приложение поддерживает русский и английский языки через res/values/strings.xml и res/values-ru/strings.xml.

3.3.1 Блок-схема алгоритма

На рисунке 8 представлена блок-схема основного пользовательского сценария — оформления заказа.

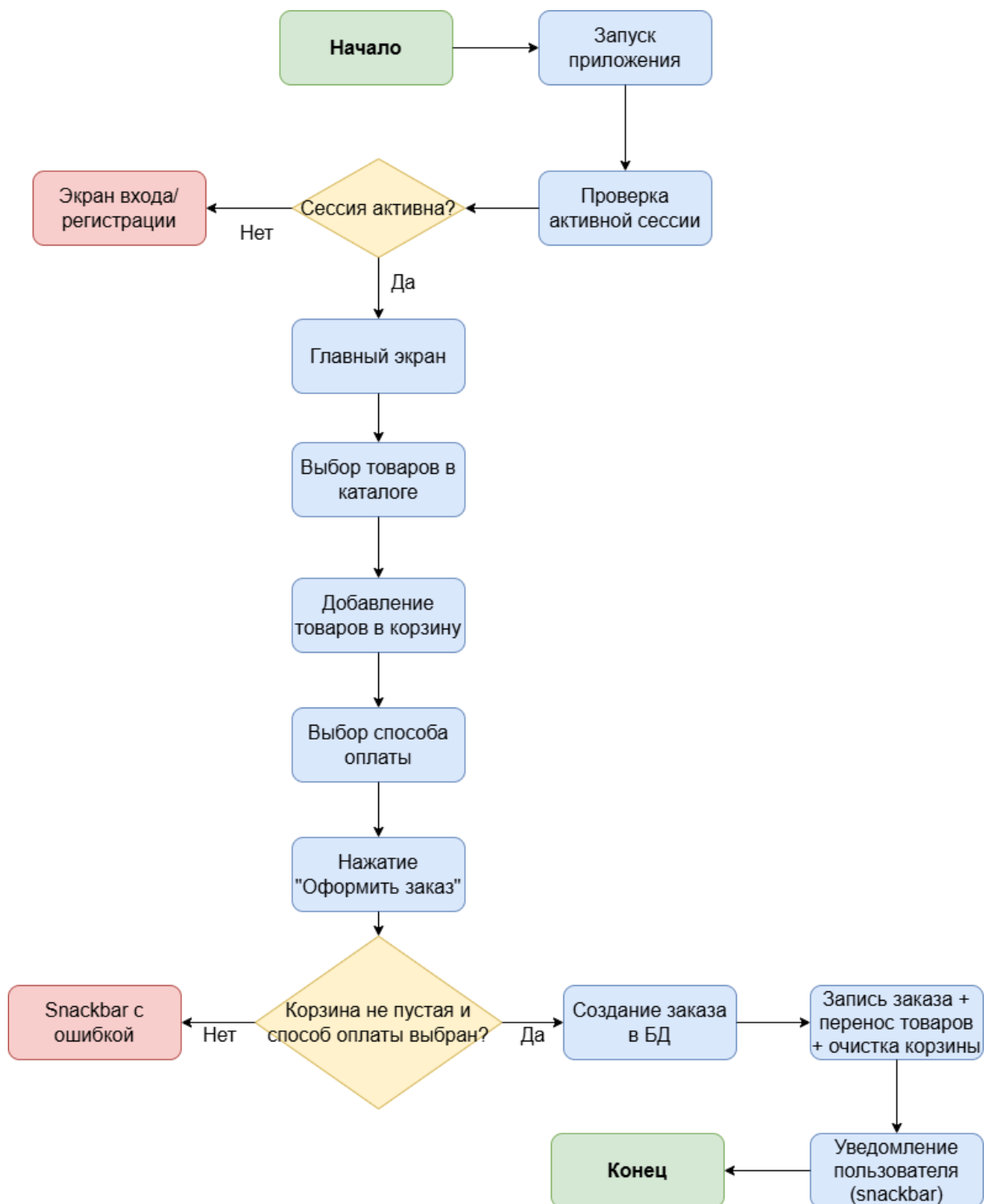


Рисунок 8 – Блок-схема пользовательского сценария оформления заказа

При старте `RegistrationActivity` проверяет сессию через `IsSessionActiveUseCase` с задержкой 1 с (для инициализации Supabase Auth). При активной сессии — сразу в `MainActivity`. После аутентификации

CatalogueViewModel загружает товары и корзину параллельно, объединяет в CatalogueUiState и публикует через LiveData.

При добавлении товара AddToCartUseCase, затем CartRepositoryImpl: два Supabase-запроса (получить quantity, INSERT или UPDATE). CartRealtimeService перехватывает событие через WebSocket и эмитирует в SharedFlow, далее оба ViewModel перезагружают корзину.

При оформлении заказа: CartViewModel проверяет непустоту корзины и наличие выбранного способа оплаты через UiState. При успешной проверке вызывает CreateNewOrderUseCase, который выполняет три последовательных операции в OrderRepositoryImpl. При успехе (isCreationSuccessful = true) Snackbar и перезагрузка корзины.

3.3.2 Диаграмма вариантов использования

Диаграмма вариантов использования (Use Case Diagram) описывает функциональные возможности системы с точки зрения взаимодействия акторов. В приложении два актора: Незарегистрированный пользователь и Зарегистрированный пользователь.

В таблице 5 представлена диаграмма вариантов использования.

Таблица 5 - Диаграмма вариантов использования

Актор	Вариант использования
Незарегистрированный пользователь	Зарегистрироваться в системе
	Войти в систему
Зарегистрированный пользователь	Просмотреть каталог товаров
	Найти товар по названию
	Отфильтровать / отсортировать товары
	Добавить товар в корзину
	Удалить товар из корзины

	Просмотреть корзину и итоговую стоимость
	Оформить заказ с выбором способа оплаты
	Просмотреть историю заказов
	Просмотреть профиль и выйти из системы

«Зарегистрироваться»: форма с 4 полями, валидация, RPC `check_phone_exists`, Supabase Auth `signUpWith`, INSERT в `users`. «Войти»: `email+пароль`, `signInWith`, получение `user_id` из таблицы `users`, `Result.Success(User)`.

«Просмотреть каталог» включает «Найти товар» (`SearchView`, фильтрация в памяти по `allProducts`) и «Отфильтровать» (`Spinner`, сортировка в памяти). Все операции выполняются локально без дополнительных запросов к серверу.

«Добавить в корзину» связан отношением `extend` с «Просмотреть корзину». «Оформить заказ» включает «Выбрать способ оплаты» как обязательный шаг.

«Просмотреть историю заказов»: `OrdersViewModel` последовательно загружает список заказов, для каждого заказа список товаров, для каждого товара информацию о продукте. Результат агрегируется в `List<OrdersUiItem>` и публикуется через `LiveData`.

Все 11 вариантов использования (2 для незарегистрированного и 9 для зарегистрированного пользователя) реализованы в полном объёме в соответствии с требованиями раздела 1.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было спроектировано и разработано мобильное приложение интернет-магазина для платформы Android, реализующее полный цикл взаимодействия покупателя с онлайн-магазином: регистрацию, аутентификацию, просмотр каталога, управление корзиной, оформление заказа и просмотр истории покупок.

Все поставленные задачи выполнены. Изучена предметная область: рассмотрены особенности жизненного цикла Android-компонентов, принципы реактивного программирования с LiveData, асинхронность через Coroutines, требования к безопасности и атомарности операций в e-commerce. Обоснован выбор Clean Architecture и MVVM как оптимального архитектурного решения.

Реализована трёхслойная архитектура: слой данных (4 репозитория, CartRealtimeService), слой предметной области (16 Use Case), слой представления (6 ViewModels, 6 экранов). DI через Koin обеспечивает слабую связанность компонентов.

Реализованные функциональные модули:

- аутентификация: регистрация с проверкой уникальности телефона через RPC PostgreSQL, вход, управление сессией;
- каталог: поиск в реальном времени, сортировка по пяти критериям, отображение количества в корзине;
- корзина: управление количеством, realtime-синхронизация через WebSocket, вычисление суммы;
- заказ: пять способов оплаты, фиксация цены на момент покупки, атомарная транзакция;
- история заказов: детализация по составу, статусу (10 вариантов), способу оплаты;
- профиль: отображение данных пользователя, выход из системы.

Технически значимые решения: realtime через Supabase Realtime с управлением lifecycle подписок; sealed class Result<T> для унификации обработки результатов; кастомный DSL-Builder для навигации; корректное управление памятью (binding = null в onDestroyView).

Пользовательский интерфейс соответствует Material Design 3. Поддерживаются русский и английский языки через ресурсные файлы. Приложение устойчиво к повороту экрана благодаря ViewModel.

Направления дальнейшего развития: административный интерфейс, push-уведомления об изменении статуса заказа, система избранных товаров, офлайн-режим с кешированием, интеграция реальных платёжных шлюзов, загрузка изображений товаров через Supabase Storage.

Цель курсовой работы достигнута. Созданное приложение демонстрирует практическое применение современного стека Android-разработки и может служить основой для коммерческого продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Supabase Documentation. Introduction [Электронный ресурс]. – Режим доступа: <https://supabase.com/docs> (дата обращения: 01.06.2026). – Текст: электронный.
2. Supabase Documentation. Realtime [Электронный ресурс]. – Режим доступа: <https://supabase.com/docs/guides/realtime> (дата обращения: 31.05.2026). – Текст: электронный.
3. Android Developers. Architecture Overview [Электронный ресурс]. – Режим доступа: <https://developer.android.com/topic/architecture> (дата обращения: 31.05.2026). – Текст: электронный.
4. Android Developers. ViewModel Overview [Электронный ресурс]. – Режим доступа: <https://developer.android.com/topic/libraries/architecture/viewmodel> (дата обращения: 31.05.2026). – Текст: электронный.
5. Android Developers. LiveData Overview [Электронный ресурс]. – Режим доступа: <https://developer.android.com/topic/libraries/architecture/livedata> (дата обращения: 31.05.2026). – Текст: электронный.
6. Kotlin Documentation. Coroutines Overview [Электронный ресурс]. – Режим доступа: <https://kotlinlang.org/docs/coroutines-overview.html> (дата обращения: 01.06.2026). – Текст: электронный.
7. Koin Documentation. Starting with Koin for Android [Электронный ресурс]. – Режим доступа: <https://insert-koin.io/docs/quickstart/android> (дата обращения: 01.06.2026). – Текст: электронный.
8. Android Developers. Dependency Injection in Android [Электронный ресурс]. – Режим доступа: <https://developer.android.com/training/dependency-injection> (дата обращения: 01.06.2026). – Текст: электронный.

9. Android Developers. Navigation Component [Электронный ресурс]. – Режим доступа: <https://developer.android.com/guide/navigation> (дата обращения: 31.05.2026). – Текст: электронный.